

```

// SOLID Principles Based E-Commerce Order Processing System

interface PaymentMethod {
    void pay(double amount);
}

class CreditCardPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}

class UPIPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using UPI");
    }
}

class WalletPayment implements PaymentMethod {
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using Wallet");
    }
}

// ISP - Separate Notification Interface
interface NotificationService {
    void sendNotification(String message);
}

class EmailNotification implements NotificationService {
    public void sendNotification(String message) {
        System.out.println("Email Sent: " + message);
    }
}

class SMSNotification implements NotificationService {
    public void sendNotification(String message) {
        System.out.println("SMS Sent: " + message);
    }
}

class PushNotification implements NotificationService {
    public void sendNotification(String message) {
        System.out.println("Push Notification: " + message);
    }
}

// ISP - Separate Storage Interface
interface OrderStorage {
    void save(Order order);
}

class DatabaseStorage implements OrderStorage {
    public void save(Order order) {
        System.out.println("Order saved to Database");
    }
}

class FileStorage implements OrderStorage {
    public void save(Order order) {
        System.out.println("Order saved to File");
    }
}

// Base Order Class
abstract class Order {
    protected int orderId;
    protected double amount;

    public Order(int orderId, double amount) {
        this.orderId = orderId;
        this.amount = amount;
    }

    public abstract double getFinalAmount();
}

class RegularOrder extends Order {

```

```

        public RegularOrder(int orderId, double amount) {
            super(orderId, amount);
        }

        @Override
        public double getFinalAmount() {
            return amount;
        }
    }

    class DiscountedOrder extends Order {
        public DiscountedOrder(int orderId, double amount) {
            super(orderId, amount);
        }

        @Override
        public double getFinalAmount() {
            return amount * 0.9;
        }
    }

    class PriorityOrder extends Order {
        public PriorityOrder(int orderId, double amount) {
            super(orderId, amount);
        }

        @Override
        public double getFinalAmount() {
            return amount + 100;
        }
    }

    // DIP - Depends on Abstractions
    class OrderService {

        private PaymentMethod paymentMethod;
        private NotificationService notificationService;
        private OrderStorage storage;

        public OrderService(
            PaymentMethod paymentMethod,
            NotificationService notificationService,
            OrderStorage storage) {

            this.paymentMethod = paymentMethod;
            this.notificationService = notificationService;
            this.storage = storage;
        }

        public void placeOrder(Order order) {

            double amount = order.getFinalAmount();

            paymentMethod.pay(amount);

            notificationService.sendNotification(
                "Order #" + order.orderId + " placed successfully"
            );

            storage.save(order);
        }
    }

    public class Main {
        public static void main(String[] args) {

            Order order = new DiscountedOrder(101, 5000);

            PaymentMethod payment = new UPIPayment();
            NotificationService notification =
                new EmailNotification();
            OrderStorage storage =
                new DatabaseStorage();

            OrderService service =
                new OrderService(
                    payment,

```

```
        notification,  
        storage  
    );  
    service.placeOrder(order);  
}
```